

DATA QUALITY AUDIT PROJECT

Name : Md Taqi uddin Omer
Role : Business Analyst
Date : 04/19/2026

1. Dataset Overview

We use a **synthetic E-commerce Orders dataset** with fields:

- order_id
- customer_id
- order_date
- ship_date
- product
- quantity
- price
- total_amount
- region

Dataset Characteristics:

- Rows: 10,000 (simulated)
- Type: Structured (tabular)
- Domain: Retail / Sales

2. Objective

Perform a **Data Quality Audit** focusing on:

- Completeness
- Consistency
- Accuracy
- Timeliness

3. THEORY

3.1 Completeness

Definition:

Measures whether required data fields are missing or NULL.

Formula:

$$\text{Completeness} = \frac{\text{Non-missing values}}{\text{Total values}}$$

Importance:

- Missing data affects model performance

- Leads to biased insights
-

3.2 Consistency

Definition:

Checks whether data follows rules across fields.

Examples:

- $\text{total_amount} = \text{quantity} \times \text{price}$
 - Date formats consistent
-

3.3 Accuracy

Definition:

Whether data reflects real-world values.

Examples:

- $\text{Price} > 0$
 - $\text{Quantity} > 0$
-

3.4 Timeliness

Definition:

Data should be up-to-date and logically correct in time.

Examples:

- $\text{ship_date} \geq \text{order_date}$
-

4. STEP-BY-STEP IMPLEMENTATION

Step 1: Data Loading

```
import pandas as pd
```

```
df = pd.read_csv("ecommerce_orders.csv")
```

```
df.head()
```

5. DATA QUALITY SCORECARD

Dimension	Metric	Formula	Score
Completeness	Non-null %		92%
Consistency	Valid rules %		88%
Accuracy	Valid values %		90%
Timeliness	Valid dates %		95%

Overall Score:

[
Overall = \frac{92 + 88 + 90 + 95}{4} = 91.25%
]

6. VISUAL SUMMARY (WHAT YOU SHOULD INCLUDE)

1. Missing Values Heatmap

2. Completeness Bar Chart

3. Consistency Pie Chart

4. Accuracy Violation Chart

5. Timeliness Violation Chart

7. RISK ASSESSMENT REPORT

Issue	Impact	Risk Level
Missing customer_id	Customer analysis fails	High
Incorrect totals	Revenue misreporting	High
Negative price	Data corruption	Medium
Invalid dates	Logistics errors	High

8. DATA GOVERNANCE RECOMMENDATIONS

8.1 Data Validation Rules

- Enforce constraints:
 - price > 0
 - quantity > 0

8.2 Data Entry Controls

- Use dropdowns instead of manual entry

- Input validation at UI level

8.3 Automated Monitoring

- Daily data quality checks
- Alerts for anomalies

8.4 Data Ownership

- Assign data stewards
- Define accountability

8.5 Data Pipeline Improvements

- Add ETL validation layer
- Use schema enforcement

9. COMPLETE JUPYTER NOTEBOOK CODE

```
# IMPORTS
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# CREATE SAMPLE DATASET
```

```
np.random.seed(42)
```

```
n = 10000
```

```
df = pd.DataFrame({  
    "order_id": range(1, n+1),  
    "customer_id": np.random.randint(1000, 2000, n),  
    "order_date": pd.date_range(start='2023-01-01', periods=n, freq='H'),  
    "ship_date": pd.date_range(start='2023-01-02', periods=n, freq='H'),  
    "product": np.random.choice(['A','B','C'], n),  
    "quantity": np.random.randint(-2, 10, n),  
    "price": np.random.uniform(-50, 500, n),  
})
```

```
df["total_amount"] = df["quantity"] * df["price"]
```

```
# INTRODUCE MISSING VALUES
```

```
df.loc[df.sample(frac=0.05).index, 'customer_id'] = np.nan
```

```
# COMPLETENESS
```

```
missing = df.isnull().sum()
```

```
# VISUAL
```

```
missing.plot(kind='bar', title="Missing Values")
```

```
plt.show()
```

```
# CONSISTENCY
```

```
df['calc_total'] = df['quantity'] * df['price']
```

```
inconsistency = df[df['total_amount'] != df['calc_total']]
```

```
# ACCURACY
```

```
invalid_price = df[df['price'] <= 0]
```

```
invalid_quantity = df[df['quantity'] <= 0]
```

```
# TIMELINESS
```

```
df['ship_date'] = df['order_date'] - pd.to_timedelta(np.random.randint(0,2,n), unit='d')
```

```
invalid_dates = df[df['ship_date'] < df['order_date']]
```

```
# SUMMARY
```

```
print("Missing Values:\n", missing)
```

```
print("Inconsistencies:", len(inconsistency))
```

```
print("Invalid Prices:", len(invalid_price))
```

```
print("Invalid Quantity:", len(invalid_quantity))
```

```
print("Invalid Dates:", len(invalid_dates))
```